



Sistemas Embebidos: estado del arte y uso

Por Juan Antonio Bonillo García

Los microcontroladores son el núcleo de la tecnología moderna: desde domótica y drones hasta sistemas críticos en aviación y robótica.

Con el auge del IoT, objetos cotidianos como bombillas se transforman en dispositivos inteligentes, haciendo la gestión de hardware más relevante que nunca.

1. Panorama Actual de Sistemas Embebidos

1.1 Microcontroladores a Medida

1.2 Propósito General

ASIC

Hardware para misión específica.
Ultra rápidos y eficientes.
Coste de diseño muy elevado.

FPGA

Reprogramable a nivel de conexiones lógicas.
Ideal para prototipado y "tiempo real".

SBCs

Microordenadores con SO (Raspberry Pi).

MCUs

Bare-Metal / RTOS.
Bajo consumo (Arduino).

Aceleradores (GPUs / NPUs)

Cálculo paralelo masivo para visión artificial y redes neuronales (Google TPU, Ethos).

"Desde domótica hasta robótica crítica, la gestión de hardware es hoy el motor de la innovación a través del IoT."

1.1.1 ASIC: Circuitos Integrados de Aplicación Específica

DEFINICIÓN Y PROPÓSITO

Hardware diseñado desde cero para una misión única. Al prescindir de lo innecesario, logran una eficiencia energética y velocidad inalcanzables por procesadores genéricos.

DSP: PROCESAMIENTO DE SEÑAL

Uso extendido en conversión Analógica-Digital. Estos procesadores (Digital Signal Processors) se basan en puertas lógicas para tareas matemáticas intensivas.

✓ Barrera de Entrada

Coste de diseño y producción masivo.
Generalmente fuera del alcance de PYMES.



1.1.2 FPGA & 1.1.3 Aplicación Práctica

FPGA: FIELD PROGRAMMABLE GATE ARRAY

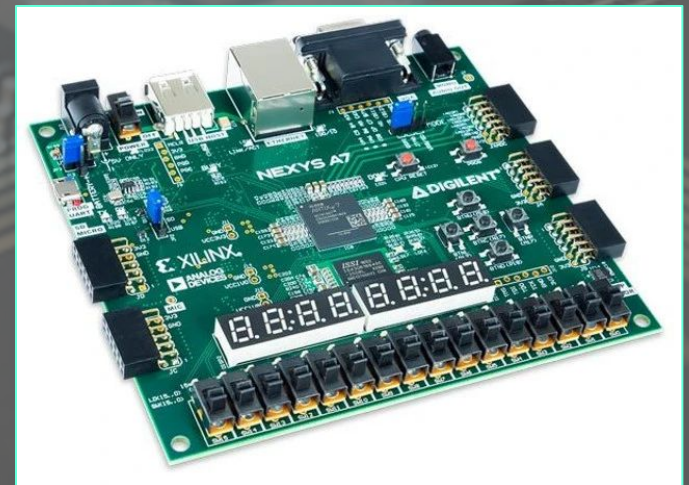
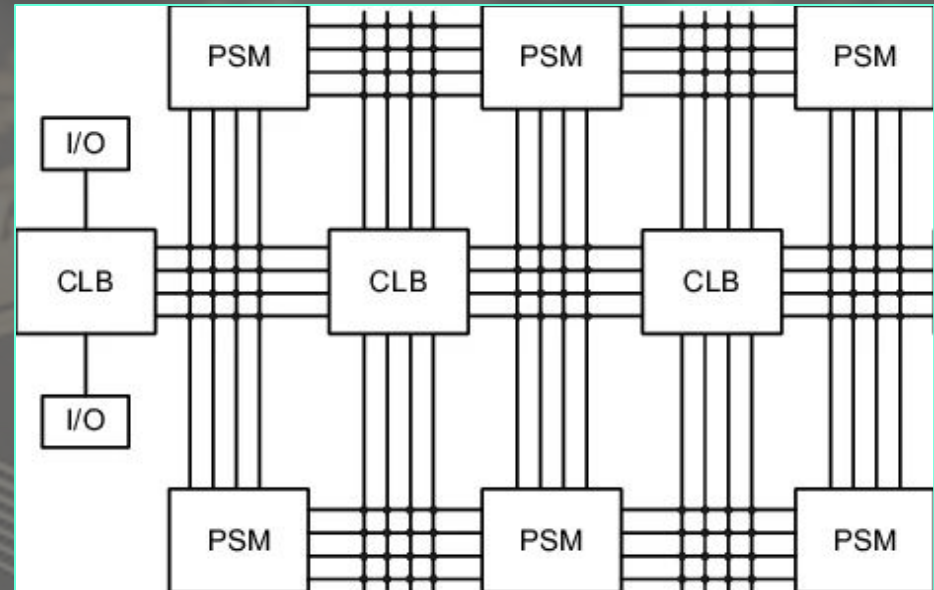
Hardware reprogramable a nivel de conexiones. Permite crear circuitos lógicos a medida utilizando puertas lógicas internas.

DESVENTAJA: Coste unitario muy elevado; rara vez viable para producción masiva.

APLICACIÓN EN LA ASIGNATURA

Estrategias de optimización para proyectos:

- **Descarga de Tareas:** Liberar al microcontrolador genérico de procesos costosos o lentos.
- **Tiempo Real:** Lograr determinismo total en la ejecución (se tratará en detalle más adelante).
- **Prototipado:** Paso previo ideal si se planea desarrollar un ASIC a futuro.



1.2.1 SBCs: Microordenadores con Sistema Operativo

POTENCIA Y VERSATILIDAD

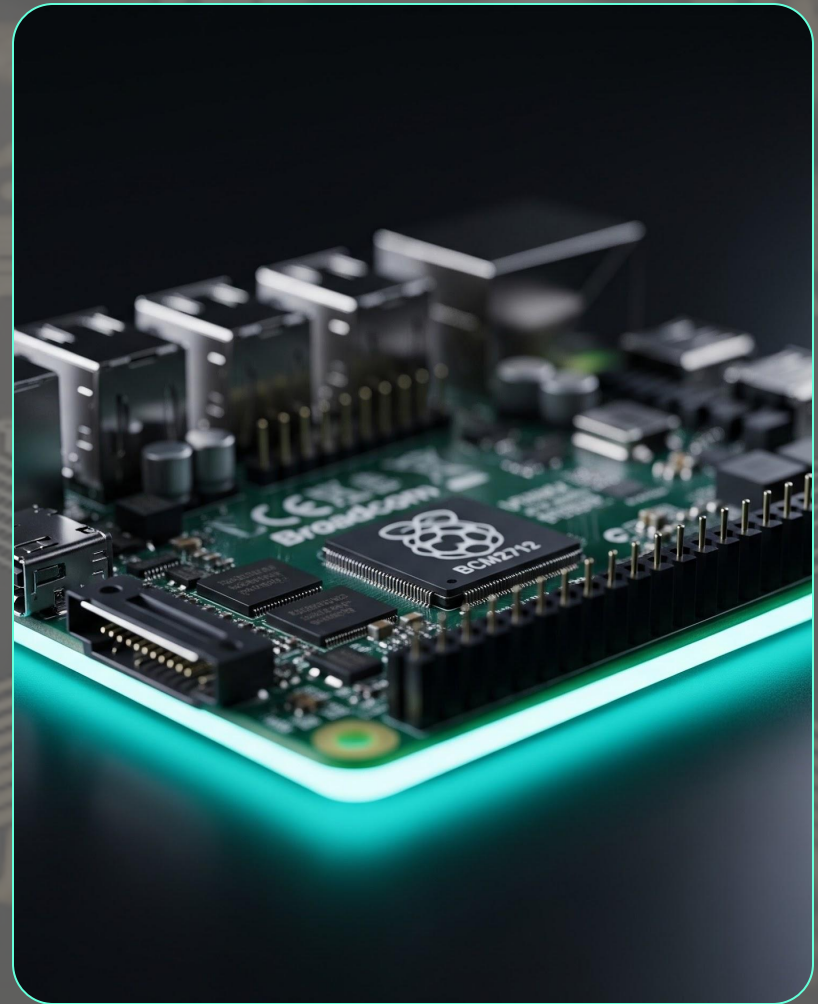
Arquitectura similar a un ordenador de sobremesa integrada en un **SoC (System on Chip)**. Ofrecen una potencia considerable por un precio reducido.

SOFTWARE Y CONECTIVIDAD

Requieren un SO completo. Su conectividad **Wi-Fi nativa** los hace piezas angulares en despliegues IoT.

CASO DE USO: NODO DE CONTROL

- Permiten procesamiento multitarea.
- Actúan como servidores para controlar microcontroladores menos potentes.
- Referencia: Raspberry Pi y equivalentes.



1.2.2 MCUs: Microcontroladores Bare-Metal / RTOS

EFICIENCIA Y COSTE

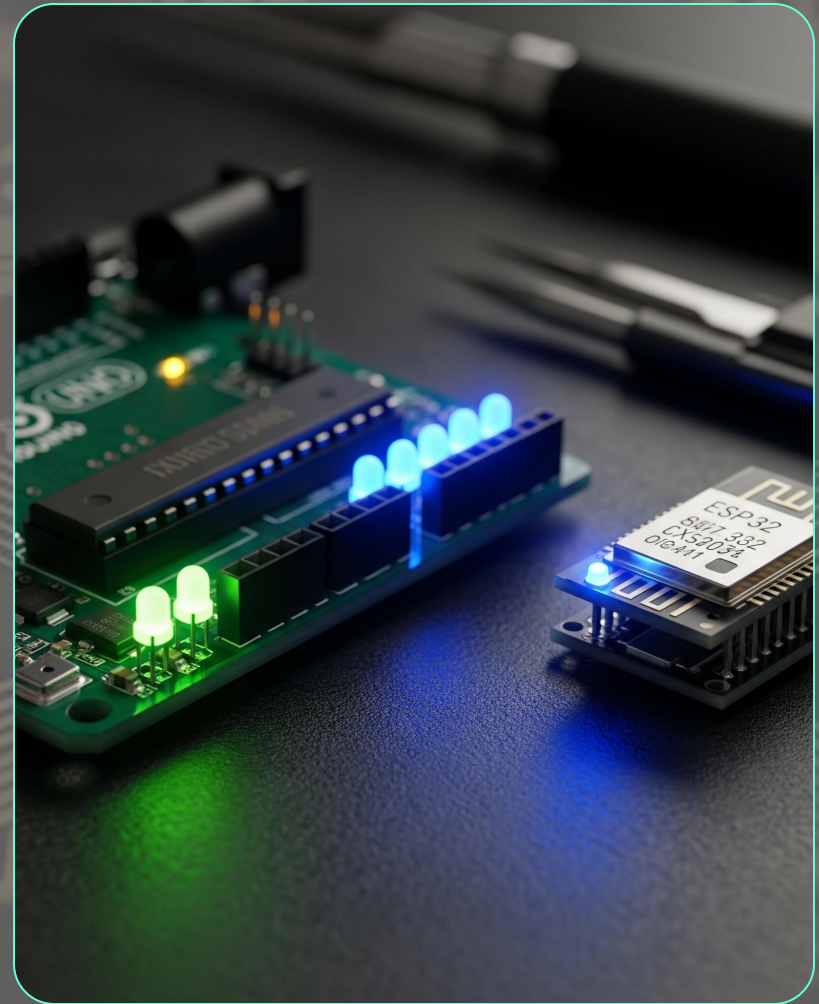
Mucho menos potentes, pero extremadamente **baratos** y con un **consumo energético mínimo**. Ideales para sensores, actuadores y hardware con baja carga de cálculo.

ARQUITECTURA DE SOFTWARE

Se trabaja habitualmente en **Bare Metal** (bucle directo en máquina sin SO) o mediante **RTOS** para gestión compleja de procesos simultáneos.

REFERENCIAS Y EJEMPLOS

- Arduino (AVR/ARM)
- ESP32 de Espressif (Wi-Fi/BT integrado)
- Nodos finales en redes IoT y dispositivos "Always-on"



1.2.3 GPUs y NPUs: Aceleración de Hardware

COMPLEMENTOS DE POTENCIA

Aunque no son microcontroladores en sí, estos coprocesadores pueden añadirse a los **SBC** para conseguir mejores prestaciones en tareas de cálculo intensivo.

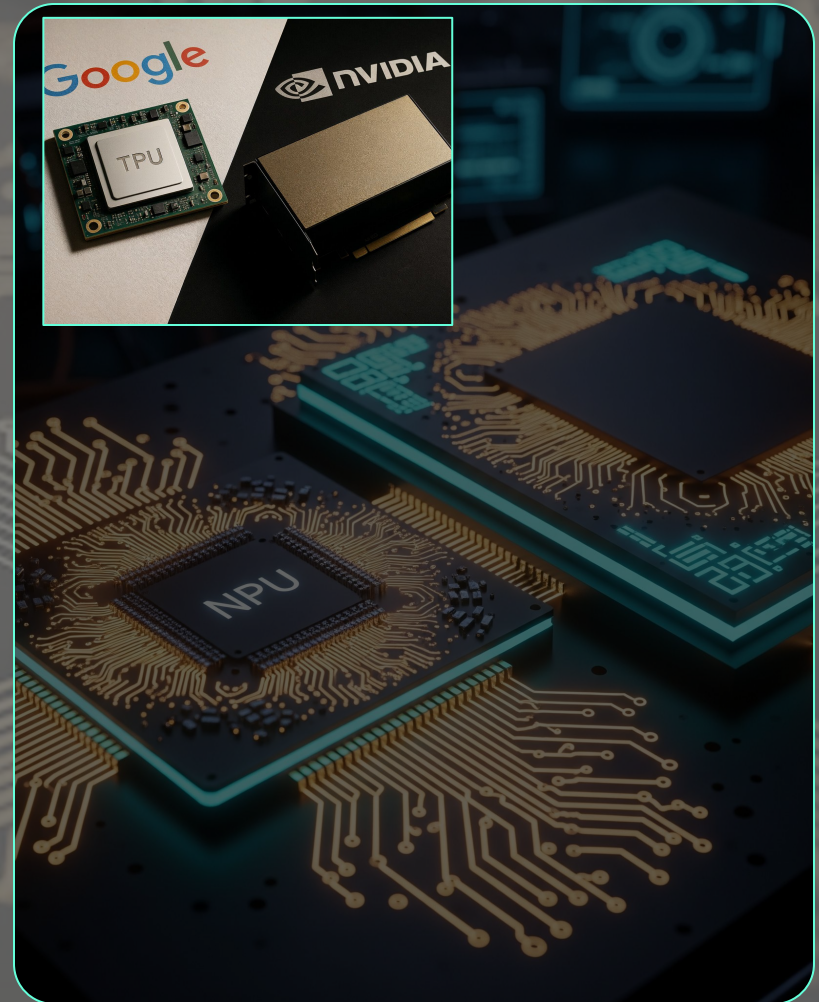
GPU (GRAPHICS PROCESSING UNIT)

Permiten un **cálculo paralelo masivo**. Son herramientas fundamentales para aplicaciones de visión artificial y procesamiento de imagen en tiempo real.

NPU (NEURAL PROCESSING UNIT)

Diseñadas específicamente para **redes neuronales** con un gasto de energía mucho más contenido que una CPU o GPU.

Ejemplos: Google TPU y ARM Ethos.



1.3 Tendencias del Mercado

1.3.1 EDGE COMPUTING

Procesado local para proteger la **privacidad**. Se envían datos procesados en lugar de imágenes en crudo.

1.3.2 TINYML

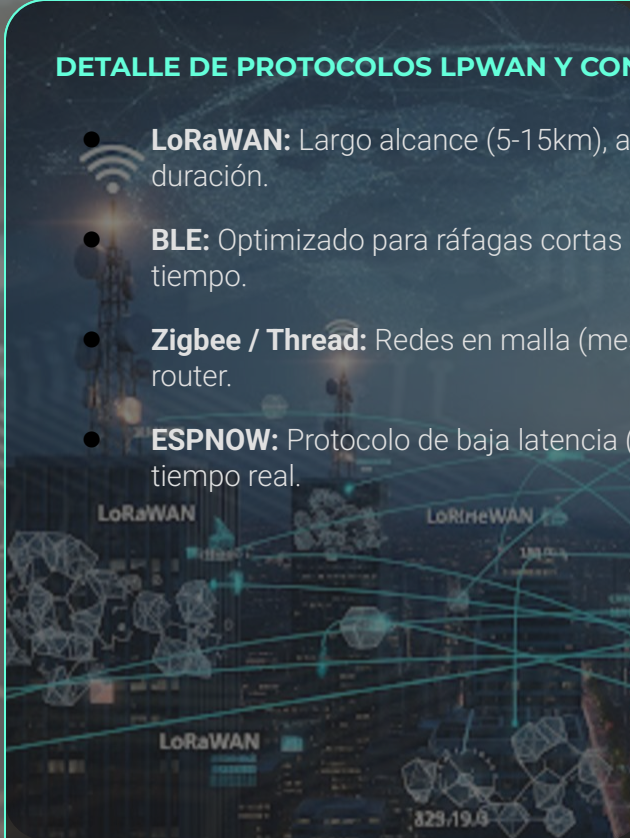
Modelos de **Machine Learning** ultra-reducidos capaces de ejecutarse en microcontroladores básicos.

1.3.3 HIPERCONECTIVIDAD (LPWAN)

Protocolos de **bajo consumo** y largo alcance para dispositivos IoT alimentados por baterías.

DETALLE DE PROTOCOLOS LPWAN Y CONECTIVIDAD

- **LoRaWAN:** Largo alcance (5-15km), alta penetración y baterías de larga duración.
- **BLE:** Optimizado para ráfagas cortas de datos; duerme el 90% del tiempo.
- **Zigbee / Thread:** Redes en malla (mesh) donde cada nodo es cliente y router.
- **ESPNow:** Protocolo de baja latencia (<10ms) sin handshakes para tiempo real.



1.3 Tendencias del Mercado

1.3.1 EDGE COMPUTING

Procesado local para proteger la **privacidad**. Se envían datos procesados en lugar de imágenes en crudo.

1.3.2 TINYML

Modelos de **Machine Learning** ultra-reducidos capaces de ejecutarse en microcontroladores básicos.

1.3.3 HIPERCONECTIVIDAD (LPWAN)

Protocolos de **bajo consumo** y largo alcance para dispositivos IoT alimentados por baterías.



1.4 Particularidades de los Microcontroladores

1.4.1 EL USO DE MQTT

Estándar para comunicar sistemas embebidos mediante una **cola de mensajería** garantizada.

Vital donde una pérdida de datos (sensor/interruptor) causaría un **error catastrófico**.

1.4.2 EL TIEMPO REAL

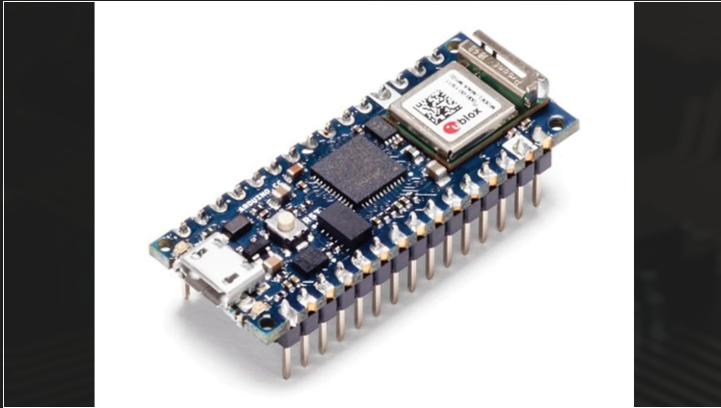
Capacidad de respuesta en un tiempo **exactamente igual** y reproducible (Determinismo).

Permite la **sincronización crítica** de sistemas físicos sin las variaciones de un SO convencional.

DETERMINISMO Y FIABILIDAD

- **MQTT vs HTTP API:** Garantía de entrega frente a la incertidumbre de pérdida de paquetes en entornos críticos.
- **Ejemplo de Seguridad:** Un sensor de puerta debe responder en 5ms exactos para evitar colisiones con humanos.
- **Bare Metal / RTOS:** Eliminan el cambio de ámbito aleatorio de los SO comerciales para garantizar tiempos de ejecución fijos.
- **Sincronización:** Base fundamental para la robótica y el control industrial avanzado.

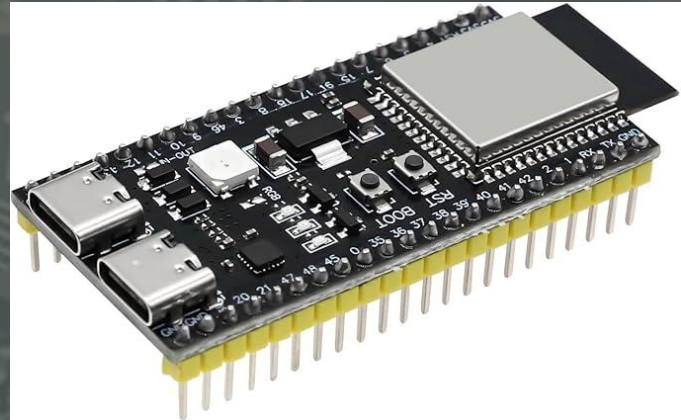
2. Hardware: Arduino vs. ESP32 en Producción



ARDUINO

Plataforma ideal para prototipado rápido y educación. Limitada en potencia de proceso y conectividad nativa en modelos básicos, pero con la mayor comunidad y ecosistema de librerías del mundo.

V
S



ESP32 (ESPRESSIF)

El estándar industrial para IoT. Ofrece Wi-Fi y Bluetooth integrados, doble núcleo y gran potencia de cálculo a un coste muy reducido. Ideal para productos finales conectados.

2.1 Ecosistema Arduino: El Gran Impulsor

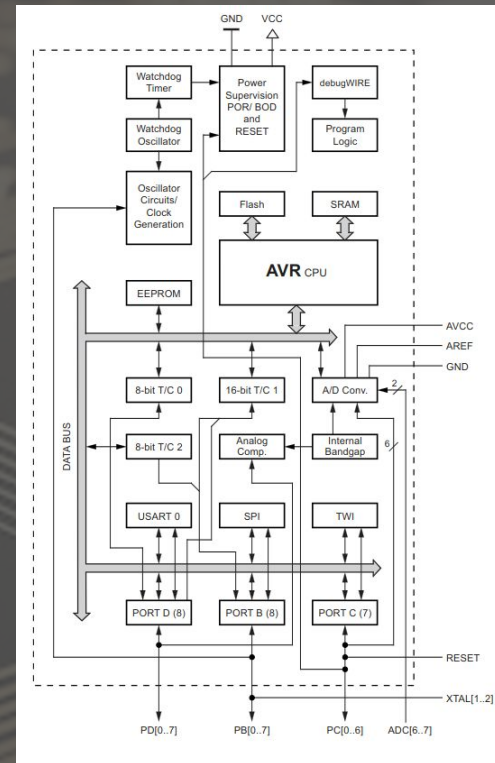
ORÍGENES Y FILOSOFÍA

Arduino democratizó el acceso a la electrónica, enfocado en aficionados, robótica y educación.

Su éxito radica en la **comunidad y simplicidad**, no en la robustez para entornos de producción.

LIMITACIONES EN PRODUCCIÓN

- Sin conectividad nativa (Wi-Fi/BT) en modelos clásicos como el UNO.
- El chip ATmega328P carece de módulos de **seguridad y bajo consumo**.
- Precio elevado frente a la potencia ofrecida.
- Eclipsado por la competencia (como ESP32) en proyectos finales industriales.

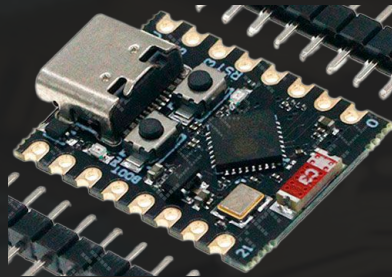


Dato actual: Las nuevas placas Arduino integran chips **ESP32** para compensar sus carencias.

2.2 Familia ESP32: El Salto Industrial

LA REVOLUCIÓN DE ESPRESSIF

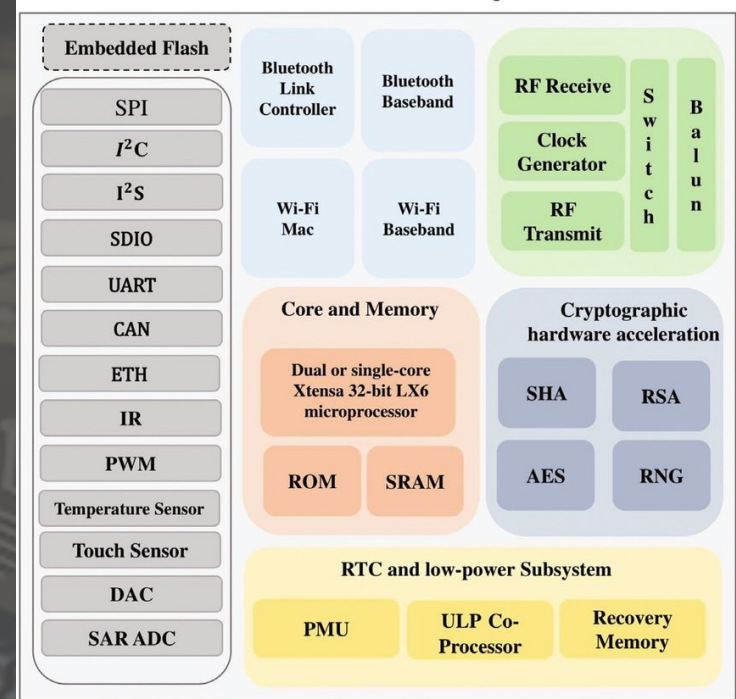
Espressif llevó el MCU a un precio y prestaciones para **producción industrial**, superando el ámbito puramente didáctico.



PUNTOS CLAVE DEL HARDWARE

- **Doble Núcleo LX6:** Verdadero paralelismo.
- **Conectividad Nativa:** Módulos para Wi-Fi y Bluetooth.
- **Seguridad Hardware:** Aceleradores AES, SHA y RSA.
- **Eficiencia Extrema:** Coprocesador ULP para bajo consumo.

ESP32 Function Block Diagram



ANÁLISIS DEL COPROCESADOR ULP

Permite pasar de mA a pocos μA en modo Deep Sleep ante eventos críticos.

Resumen Técnico Comparativo

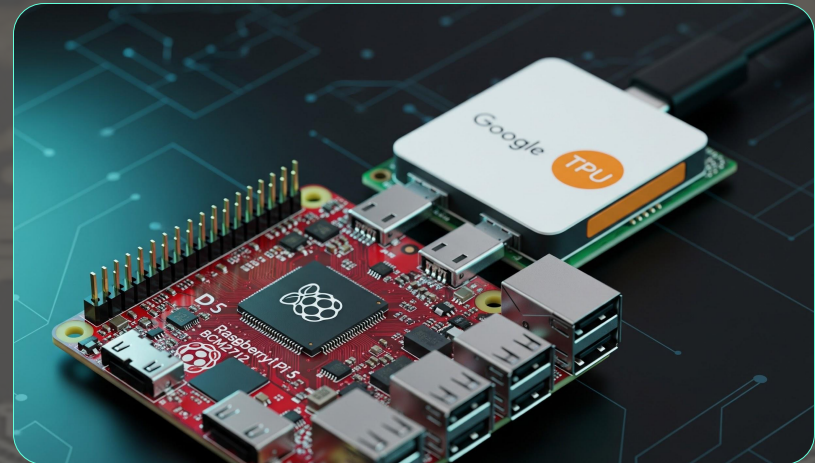
Característica	Arduino UNO (ATmega328P)	ESP32 (Xtensa Dual-Core)
Arquitectura	8-bits (Un núcleo)	32-bits (Doble núcleo)
Velocidad Reloj	16 MHz	160 - 240 MHz
Memoria RAM	2 KB	520 KB
Conectividad	Ninguna nativa	Wi-Fi + Bluetooth (Classic/BLE)
Seguridad HW	No	AES, SHA, RSA
Precio (Chip)	~2.50 €	~1.50 €

2.3 Microordenadores: El Salto al Edge

RASPBERRY PI Y TINYML

Cuando el **tiempo real estricto** no es el factor limitante, estos sistemas ofrecen la potencia necesaria para **Edge Computing** avanzado.

Permiten ejecutar modelos de IA complejos que un microcontrolador estándar no podría procesar eficientemente.



HARDWARE DEDICADO Y ACCELERACIÓN

- **Capacidad de Proceso:** El uso de Microordenadores permite gestionar sistemas operativos completos (Linux), facilitando el desarrollo en Python y C++.
- **Google TPU (Coral):** Integración de hardware específico para inferencia de redes neuronales, multiplicando la velocidad de ejecución.
- **Casos de Uso:** Visión artificial avanzada, procesamiento de lenguaje natural en local y pasarelas IoT de alta densidad de datos.
- **Versatilidad:** Ideal para actuar como cerebro central en sistemas donde la latencia de milisegundos no es crítica pero la inteligencia sí.

4. Programación: Del Bucle al RTOS

EL RETO: BLINK ADAPTATIVO

Diseñar un sistema de control de parpadeo mediante pulsador:

- **0 pulsaciones:** LED apagado.
- **1 a 5 pulsaciones:** Velocidad incremental.
- **6ª pulsación:** Reinicio del ciclo.

APROXIMACIÓN SECUENCIAL

Modelo tradicional de Arduino (hilo único).

El problema del "Bloqueo":

Si usamos `delay()`, el procesador se detiene. Las pulsaciones se pierden porque el sistema está "ciego" durante la espera.

Solución: Polling (millis)

Escanear el botón constantemente y usar marcas de tiempo para la lógica del LED sin detener la ejecución.

CÓDIGO LOOP (SIMULACIÓN PARALELISMO)

```
#include <Arduino.h>

const int LED_PIN = 2;

const int BTN_PIN = 4;

int nivelVelocidad = 0;

unsigned long ultimoParpadeo = 0;

unsigned long ultimaPulsacion = 0;

bool estadoLed = false;

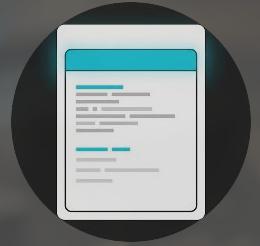
void setup() {
    pinMode(LED_PIN, OUTPUT);

    pinMode(BTN_PIN, INPUT_PULLUP); // Resistencia interna activa
}

void loop() {
    // 1. ESCANEO DEL BOTÓN (Debouncing manual)
    if (digitalRead(BTN_PIN) == LOW && (millis() - ultimaPulsacion > 200)) {
        nivelVelocidad = (nivelVelocidad + 1) % 6;
        ultimaPulsacion = millis();
    }

    // 2. LÓGICA DEL LED (Sin usar delay)
    if (nivelVelocidad > 0) {
        int intervalo = 1000 / nivelVelocidad; // A más nivel, menos tiempo (más rápido)
        if (millis() - ultimoParpadeo >= intervalo) {
            estadoLed = !estadoLed;
            digitalWrite(LED_PIN, estadoLed);
            ultimoParpadeo = millis();
        }
    } else {
        digitalWrite(LED_PIN, LOW);
    }
}
```


Desglose del Código Secuencial



2.1 SECCIÓN DE CABECERA

Casi idéntico a cualquier programa de C++, incluimos las librerías necesarias para el funcionamiento del sistema.

- **Librerías:** Includes básicos y específicos para sensores o actuadores.

DEFINICIÓN DE CONSTANTES

Para evitar el uso de "**números mágicos**" a lo long del código, definimos las constantes y variables globales aquí.

- **Pines:** Asignación física de E/S.
- **Estados:** Variables para control de tiempo (millis) y lógica del LED.

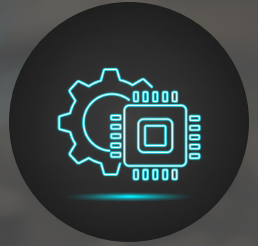
CABECERA Y VARIABLES

```
// Librerías principales
#include <Arduino.h>

// Definición de hardware
const int LED_PIN = 2;
const int BTN_PIN = 4;

// Variables de control global
int nivelVelocidad = 0;
unsigned long ultimoParpadeo = 0;
unsigned long ultimaPulsacion = 0;
bool estadoLed = false;
```

Desglose del Código: Configuración



2.2 SECCIÓN DE CONFIGURACIÓN (SETUP)

En esta sección definiremos todo lo necesario para que los sensores y actuadores funcionen correctamente.

Esta sección es crítica para el ciclo de vida del hardware:

- **Ejecución única:** Solo se ejecuta una vez al encender o reiniciar el microcontrolador.
- **Inicialización:** Ideal para configurar modos de pines, comunicaciones seriales o librerías de sensores.

FUNCIÓN VOID SETUP()

```
void setup() {  
    pinMode(LED_PIN, OUTPUT);  
    pinMode(BTN_PIN, INPUT_PULLUP);  
    // Resistencia interna activa  
}
```


Desglose del Código: El Bucle Infinito



2.3 EL MOTOR DEL PROGRAMA (LOOP)

Esta es la parte del código que se repite una y otra vez mientras el microcontrolador esté encendido.

Características fundamentales:

- **Persistencia:** Mantiene la lógica del sistema activa de forma indefinida.
- **Secuencialidad:** El código se ejecuta línea a línea. Si una parte bloquea el hilo, el resto del motor se detiene.
- **Modularidad:** Permite llamar a funciones externas para mantener el código organizado.

FUNCIÓN VOID LOOP()

```
void loop() {  
    // 1. ESCANEO DEL BOTÓN  
    if (digitalRead(BTN_PIN) == LOW &&  
        (millis() - ultimaPulsacion > 200)) {  
        nivelVelocidad = (nivelVelocidad + 1) % 6;  
        ultimaPulsacion = millis();  
    }  
    // 2. LÓGICA DEL LED  
    if (nivelVelocidad > 0) {  
        int intervalo = 1000 / nivelVelocidad;  
        if (millis() - ultimoParpadeo >= intervalo) {  
            estadoLed = !estadoLed;  
            digitalWrite(LED_PIN, estadoLed);  
        } else { digitalWrite(LED_PIN, LOW); }  
    }  
}
```

Programación Avanzada: FreeRTOS

3. APROXIMACIÓN PROFESIONAL

En un ESP32, el uso de FreeRTOS permite crear tareas independientes que se ejecutan "simultáneamente" delegando el flujo del programa.

- **Tarea de Control:** Vigila el botón y actualiza variables.
- **Tarea de Actuación:** Controla el parpadeo del LED.
- **Eficiencia:** `vTaskDelay` pausa la tarea, no el procesador.

MENTALIDAD DE INGENIERÍA

Con RTOS, la función `loop()` queda libre o vacía. El sistema operativo en tiempo real gestiona las prioridades y el tiempo de CPU.

Las tareas pueden ser bucles infinitos individuales o procesos de un solo uso que se destruyen tras ejecutarse.

CÓDIGO MULTITAREA (RTOS)

```
volatile int nivelGlobal = 0;

// TAREA 1: BOTÓN
void tareaBoton(void *pvP) {
    for (;;) {
        if (digitalRead(BTN_PIN) == LOW) {
            nivelGlobal = (nivelGlobal + 1) % 6;
            vTaskDelay(pdMS_TO_TICKS(250));
        }
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

// SETUP: CREACIÓN
void setup() {
    pinMode(LED_PIN, OUTPUT);
    pinMode(BTN_PIN, INPUT_PULLUP);
    xTaskCreate(tareaBoton, "Btn", 2048, NULL, 1, NULL);
    xTaskCreate(tareaLed, "LED", 2048, NULL, 1, NULL);
}

// LOOP: LIBRE
void loop() { vTaskDelete(NULL); }
```


4. Análisis: El Estándar Industrial



DETERMINISMO GARANTIZADO

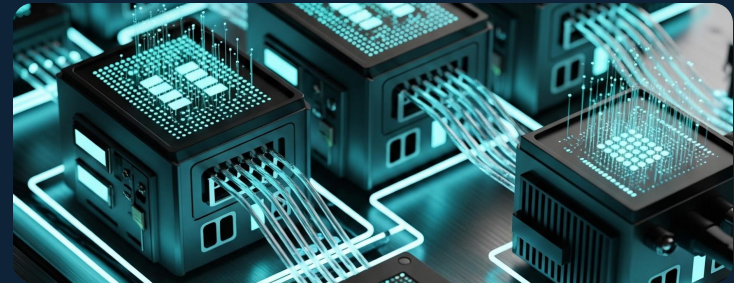
A diferencia del `loop()`, donde una función lenta bloquea todo el sistema, RTOS asegura que las tareas críticas (como un botón de emergencia) tengan su tiempo de CPU reservado por el planificador.

EFICIENCIA ENERGÉTICA

El `delay()` tradicional quema ciclos al 100% de potencia. En RTOS, `vTaskDelay()` permite que el procesador entre en modo "Low Power" mientras espera, optimizando el consumo.

ESCALABILIDAD MODULAR

Añadir componentes (como una pantalla OLED o conectividad Wi-Fi) no requiere reescribir el código existente. Se crea una nueva tarea independiente que convive sin interferencias.



CONCLUSIÓN TÉCNICA

Para dispositivos IoT que gestionan comunicaciones (Wi-Fi/BT) y eventos físicos en paralelo, el uso de un RTOS es **obligatorio** para garantizar la respuesta en tiempo real y la fiabilidad industrial.

1. Comparativa: Arduino IDE vs. PlatformIO



Mientras uno es un **"bloc de notas con esteroides"**, el otro es un sistema de gestión de ingeniería profesional para el desarrollo de firmware.

Característica	Arduino IDE 2.0	PlatformIO (VS Code)
Instalación	Muy simple (Standalone)	Extensión de VS Code
Librerías	Global (Todo el sistema)	Local (Aislada por proyecto)
Autocompletado	Básico	Avanzado (IntelliSense)
Versiones	Manual / Externo	Integración total con Git
Debugging	Limitado (Serial)	Avanzado (Breakpoints)
Soporte Placas	Una a la vez	Varias placas simultáneas

2. Beneficios y Desventajas



2.1 Arduino IDE

BENEFICIOS

- **Ideal para principiantes:** No requiere configurar archivos complejos.
- Si quieres encender un LED en 30 segundos, es tu herramienta.

DESVENTAJAS

- **Inviabile para proyectos grandes:** La gestión global de librerías es un riesgo.
- Actualizar una librería puede **"romper"** sin querer el código de proyectos antiguos.

2.2 PlatformIO

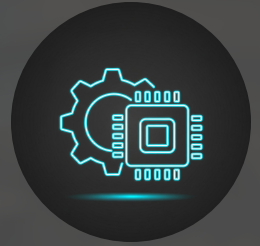
BENEFICIOS

- **Profesionalismo total:** Proyectos autocontenidos gracias a **platformio.ini**.
- Garantía de funcionamiento: las librerías están fijadas por proyecto para el futuro.

DESVENTAJAS

- **Curva de aprendizaje:** Requiere entender rutas de archivos y configuración.
- Entrada inicial más compleja para usuarios novatos.

3. Mini-Tutorial: Mi primer "Blink"



Caso A: Arduino IDE

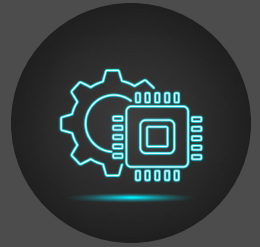
- **1.** Conecta tu placa al puerto USB.
- **2.** Selecciona tu modelo en **Herramientas > Placa**.
- **3.** Escribe el código en el editor principal.
- **4.** Pulsa el botón de la **Flecha (Subir)** para cargar el programa.

CÓDIGO FUENTE (C++)

```
// Configuración inicial
void setup() {
  pinMode(2, OUTPUT);
}

// Bucle infinito
void loop() {
  digitalWrite(2, HIGH); // ON
  delay(1000);
  digitalWrite(2, LOW); // OFF
  delay(1000);
}
```

3. Mini-Tutorial: Mi primer "Blink"



Caso B: PlatformIO (VS Code)

1. Crea un **New Project** en el Home de PlatformIO.
2. Configura: Selecciona tu placa (ej. ESP32 Dev) y Framework (Arduino).
3. Gestión de archivos: Abre la carpeta **src** y busca el archivo **main.cpp**.
4. Añade la cabecera: Debes incluir **#include <Arduino.h>** al principio.
5. Compila y Sube: Usa los iconos de **Flecha** o **Check** en la barra azul.

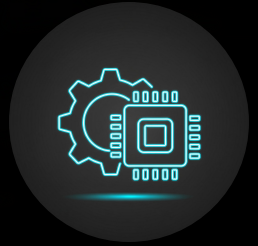
CÓDIGO FUENTE (main.cpp)

```
#include <Arduino.h>

// Configuración
void setup() {
    pinMode(2, OUTPUT);
}

// Ejecución
void loop() {
    digitalWrite(2, HIGH);
    delay(1000);
    digitalWrite(2, LOW);
    delay(1000);
}
```


4. Conclusiones



Usa Arduino IDE si...

Ideal para la fase inicial y pruebas de concepto rápidas.

Estás **aprendiendo las bases** de la programación electrónica.

Vas a realizar un proyecto sencillo de **un solo archivo**.

Necesitas **probar un sensor** o actuador rápidamente.

Usa PlatformIO si...

La herramienta estándar para ingeniería de firmware profesional.

Desarrollas un **producto final** para vender o entregar.

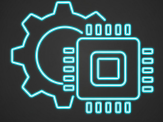
Trabajas en **equipo** y utilizas control de versiones (Git).

Tu proyecto escala y usa **más de 2 librerías** externas.

VEREDICTO FINAL

PlatformIO es la herramienta que te define como **profesional**.

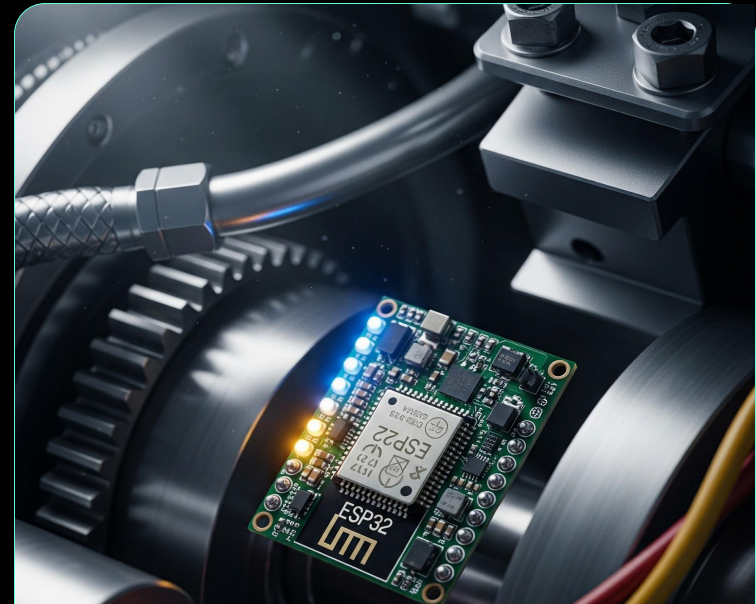
5. El Enfoque IMCR



Mantenimiento en Producción

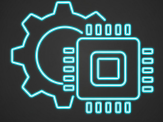
El diseño profesional no termina cuando el código compila. Una vez desplegado en **entornos reales**, el acceso físico es un desafío crítico.

- Dispositivos en altura (10m+) o naves industriales.
- Instalaciones en interior de maquinaria pesada.
- Costes logísticos **extremadamente altos** por desplazamiento técnico.



Ingeniería **tolerante a fallos**: recuperación autónoma y actualización remota sin intervención física.

5.1 Mantenimiento Autónomo: Watchdog Timer (WDT)



¿Cómo funciona el "Perro Guardián"?

Es un mecanismo de **seguridad por hardware** independiente del núcleo principal que evita bloqueos permanentes.

- **Cuenta regresiva:** Un temporizador físico que corre continuamente hacia cero.
- **Alimentar al perro:** El software debe reiniciar el contador periódicamente ("kick the dog").
- **Reinicio Automático:** Si el sistema se bloquea (bucle infinito, fallo Wi-Fi), el WDT llega a cero y fuerza un **Reset eléctrico**.

Garantiza operatividad en segundos **sin intervención humana**.



Estrategia de **mantenimiento correctivo autónomo**: el hardware vigila al software para asegurar la resiliencia total.

5.2 Mantenimiento Evolutivo: Actualizaciones OTA

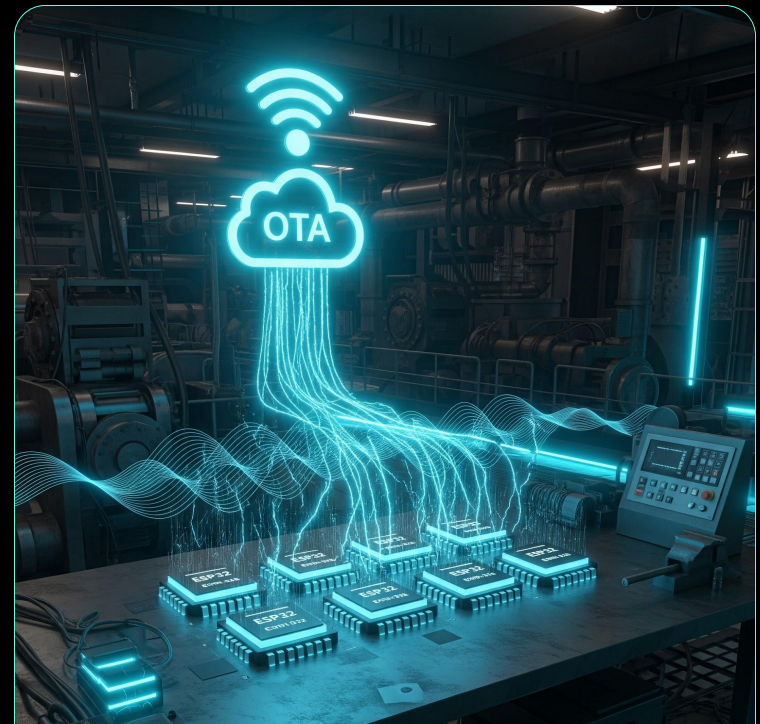


Evolución y Despliegue Remoto

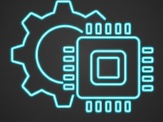
Añade funcionalidades y corrige vulnerabilidades sin retirar el equipo de la planta.

- **Over-The-Air (OTA):** Actualización inalámbrica y cifrada vía Wi-Fi o Nube.
- **Doble Partición:** El firmware se escribe en una partición secundaria mientras la original sigue activa.
- **Rollback Seguro:** Si el nuevo código falla, el sistema **revierte cambios** volviendo a la versión anterior.
- **Escalabilidad:** Actualiza flotas de miles de dispositivos con un solo clic.

Reduce drásticamente costes logísticos frente al método tradicional por cable.



Actualización de flotas **globales**: seguridad y nuevas funciones desplegadas de forma masiva y **sin cables**.



¿Dudas o preguntas?

¡Muchas gracias por vuestra atención!

